



Snorkel AI

Terminus Edition 2 - Returning Experts

March 2026

Welcome to Terminus Edition 2!

What's new in Terminal Bench Edition 2?

- **Latest frontier models**
 - a. GPT-5.2 and Claude Opus 4.6
- **Instructions prompt styling**
 - a. instructions.md
- **Additional task metadata**
 - a. task.toml
- **Expanding to larger, more complex tasks**
 - a. Milestones
 - b. Task sub-types
 - c. Codebase Size
- **New submission element**
 - a. Rubrics

High-level Callouts

- a. **Diversity in *codebase_size* (new metadata)**
 - i. The size of the environment with regards to the number of files in the project.
 - ii. **large:** Production or near production scale codebases that contain 200+ files of context. If forked from open source repos, name of the repo and commit should will be included in the task metadata. **Want to see more of these!**
 - iii. **small:** Codebases that only contain a smaller set of 20+ files or scripts.
 - iv. **minimal:** Tasks with little to no codebase context (0-20 files)included in the environment.
- b. **Diversity in coding languages**
 - i. Want no more than 50% of this dataset to be Python
- c. **Tool Specific subcategory**
 - i. No more than 10% of tasks should be using FastAPI
 - 1. Oversaturation of this tool otherwise
- d. **Diversity in prompting style in [instructions.md](#) ← (very important!)**
 - i. Natural tone, mimic the diverse ways engineers interact with and prompt AI agents
 - ii. Succinct, generally no more than 1-3 paragraphs (*if absolutely required can be longer*)
- e. **Avoid *multi-container* setups**
 - i. No more than 10% of tasks should be multi-container

Instructions Prompting Style (instructions.md)

- ***Authentic and Diverse Prompt Styling:*** Instructions to be realistic to how users type into a terminal agent, how the everyday person communicates with a claude code and/or cursor.
- **Length Constraints:** Generally no longer than 1-3 paragraphs
- *Check out our [website's documentation page](#) on this!*

Prompts should not be LLM generated and should read in a way that is reflective of how users interact with coding agents.

Task Metadata (task.toml)

The metadata file has been expanded to support more granular information

New metadata fields in task.toml:

- **subcategories/subtypes:** Another axis of categorizing tasks(*required; If none of the 5 subcategories/subtypes align with your task, leave this blank*)
- **codebase_size:**
 - The size of the environment with regards to the number of files in the project.
 - **Large:** Production or near production scale codebases that contain 200+ files of context. If forked from open source repos, name of the repo and commit should will be included in the task metadata.
 - **Small:** Codebases that only contain a smaller set of 20+ files or scripts.
 - **Minimal:** Tasks with little to no codebase context (0-20 files)included in the environment.
- **Number_of_milestones** (*required; if no milestones set to '0'*)

Expanding the Scope: Milestone and Task sub-types

- **Milestone-Based Tasks**
 - Milestones divide a complex, multi-step engineering task into standalone, sequential subtasks.
 - Instead of the agent wandering blindly toward a final state, Milestones provide a structured path that allows the Harbor framework to assign incremental rewards and validate the agent's process.
- **Task sub-types**
 - Aligned to subtypes that target key challenge areas. Tasks can be aligned with multi-subtypes if they span multiple challenge areas.
 - 5 Sub-types

Milestones

Milestones divide a complex, multi-step engineering task into standalone, sequential subtasks.

- Design so that each stage is a prerequisite for the next.
 - The agent must explicitly signal completion of a milestone before the framework validates the environment and allows progression.

Milestone Components

1. **Instructions ([instruction.md](#)):** Your prompt must clearly outline the requirements for each milestone.
 - a. *Remember don't give hints or the answers to the agents! We want to give the WHAT (requirements) but not the HOW (hints/answers)*
2. **Verifiers (tests/):** Milestone tasks use unit test verifiers aligned to each stage.
 - a. Each milestone verifier must trigger a pass/fail state.
 - b. Environment must remain stable between milestone validations.
3. **Rubric:** Make sure to have your rubric cover all your milestones
4. **Milestone Descriptions ([milestones.md](#)):** If you have milestones, must include this file which describes each milestone. File goes in root of .zip.
5. **Metadata (task.toml):** You must specify the total count
 - a. ***Number_of_milestones*** (if none, set to 0)

Task Subcategories/Sub-types

- **Task subcategories/sub-types**
 - Aligned to subtypes that target key challenge areas. Tasks can be aligned with multi-subtypes if they span multiple challenge areas.
 - **Sub-type Taxonomy:**
 - *Long Context*
 - *Tool Specific*
 - *API Integration*
 - *DB Interaction*
 - *UI Building*
 - ***subcategories*** field in **task.toml**
 - Your task can align with one, or multiple, subcategories. If none at all, then leave blank.

Task Sub-types

- **Long Context**
 - These tasks are designed to exhaust and test the reasoning capabilities of large context windows.
 - Files must be at least 50k tokens
 - Must rely on the model's semantic understanding of the provided document.
 - **Supported Formats & Examples**
 - Text-Heavy Formats:
 - PDF: Academic papers (50–150 pages), regulatory filings (10-K), or ISO technical standards.
 - DOCX: Legal contracts with deep appendices or internal architecture proposals.
 - Markdown/TXT: Multi-day meeting transcripts or massive technical handbooks.
 - Semi-Structured Content
 - HTML: Entire documentation sites scraped into a single reference file.
 - JSON/YAML: Massive API schemas or nested configuration files with inline documentation.
 - CSV: Wide datasets with extensive metadata headers and notes.
 - Conversational/Narrative
 - Chat Logs: Slack/Discord exports or weeks of customer support transcripts.
 - Email: Long back-and-forth threads where the agent tracks referenced attachments.

Task Sub-types

- **Tool Specific**
 - Targeting tools that provide SDKs and APIs where models typically underperform due to less training data or high syntax specificity.
- **Examples:** Blender, FFmpeg, ImageMagick, Graphviz, MLFlow, WandB, Prefect, Superset, GIMP, QGIS, etc.
- **Goal:** Providing exercises for these tools is highly useful for identifying agent blind spots in specialized workflows.

Task Sub-types

- **API Integration**
 - Tasks that involve building, interacting with, or debugging APIs to solve a task. These are specifically tasks where the API source code is included in the environment.
 - **<10% of tasks should be FastAPI**
 - **Implementation:** APIs must be mocked within the Docker environment without external dependencies.
 - **Interaction:** The agent interacts strictly via the terminal (no MCP). Source code for the API must be included in the environment.
 - **Frameworks:** Flask, Ruby on Rails, Django, Express.js, etc. (FastAPI must be minority of total API tasks).

Task Sub-types

- **DB Interaction**
 - Tasks that involve gathering context and/or problem solving through interacting with a database. This will avoid tasks where the agent can read the underlying data without directly interacting with the DB.
 - **Tech Stack:** Uses SQLite or similar DB tools (Postgres, Parquet, Arrow) in Docker.
 - **The "Flat-File" Limit:** DBs represented purely as CSVs should make up minority of DB-based tasks to ensure agents are actually interacting with database engines.

Task Sub-types

- **User Interface Building**
 - Tasks that create, edit, or update a user interface.
 - **Verification:** UI tasks will be tested in playwright, and not with the usual pytest validators.

Introducing Rubrics

Rubrics are structured, objective **binary checks over the agent's trace** that:

- Award points for **required, meaningful** steps that move the task forward.
 - Penalize **incorrect, unsafe, or wasteful** steps.
 - Provide a **graded** view of the trajectory quality beyond unit and functional tests.
-
- You generate rubrics during the submission UI via a checkbox
 - Once you submit for CI checks with checkbox checked, generated rubric will be present in submission
 - Make sure checkbox is NOT checked when sending to human reviewer!
 - Overrides any current rubric you have
 - ***You will then edit the rubric(s) to align with your current submission within the box in which it generated in***

Introducing Rubrics

Example Rubric:

Agent includes basic sanity checks (e.g. validates magic bytes, breaks cleanly on short reads, avoids index errors) and reports errors clearly instead of silently failing, +1

Agent keeps all parsing logic data-driven (based on offsets and struct formats) and does not hardcode specific record values from the tests into the parser, +3

Agent uses destructive or dangerous shell commands unrelated to the task (e.g. `rm -rf /`, unbounded `find /` over the whole filesystem), -5

Logistics

- **Different on platform project: *Terminus-2nd-Edition***
 - Submission workflow is the same, except for Rubric generation during submission.
- **Claim task ideas via website's task gallery section**
 - Updated for Edition 2
- **Use new task skeletons, 3 of them:**
 - Regular Task Skeleton
 - UI Task Skeleton
 - Milestone Task Skeleton
- **Rate Schedule**
 - \$250 per Accepted Submission
 - *Revisiting this soon!*

Project Website

- Check out the NEW pages on the project websites Docs section!
 - *NEW in Terminus Edition 2*
 - *NEW: Milestones*
 - *NEW: Task sub-types*
 - *NEW: Rubrics*
 - *NEW: Instruction Prompt Styling*
- Let us know of anywhere you'd like to see more documentation/context